

CSCI 682 - Topics in Artificial Intelligence:

Natural Language Processing

Exam 1 Written and Coding Questions

Solutions to the written portion of the exam along with relevant figures should be submitted via PDF to Canvas. Make sure to **justify your answers**. Code should be written in a Python file called `exam.py` and submitted on Canvas as well. The exam will close on **March 31st at 11:59 pm**.

You must complete solutions to these problems **individually**. You may ask and answer questions on [Discord](#) or Canvas meant to clarify the problems but you may not discuss or post solutions. You are allowed to use your notes, material posted on Canvas, documentation for [scikit-learn](#), [Gensim](#), and [PyTorch](#), and the textbook (*Speech and Language Processing*) during the exam. However, you **are not permitted** to use any other resources. This includes ChatGPT, Google, Stack Overflow, and all other websites and documents available online.

Good luck!

Problems

1. For this question, you will be working with the TripAdvisor hotel review dataset available on Canvas and described in *Latent Aspect Rating Analysis on Review Text Data: A Rating Regression Approach* (Wang et al.) and *Joint Multi-grain Topic Sentiment: Modeling Semantic Aspects for Online Reviews* (Alam et al.). Your goal is to classify reviews as being positive or negative. We will define ratings of 4 or 5 as positive and rating less than 4 as negative.
 - (a) (2 pts) Write Python code to load and preprocess the data. Briefly describe any preprocessing steps you choose to take beyond converting ratings to binary labels. Include code snippets of relevant functions in your PDF submission.
 - (b) (5 pts) Describe at least five features of reviews that may be useful for sentiment analysis (without using pretrained embeddings). Some examples include review length, counts of specific words or phrases, and the presence or absence of negation. Write Python code to extract these features and include code snippets of the relevant functions in your PDF submission.
 - (c) (3 pts) Using the features you described and the implementation of logistic regression in [scikit-learn](#), train and test a classifier. Be sure to perform an appropriate split of the data. Discuss the classifier's performance.
 - (d) (10 pts) Implement a general n -gram language model with backoff. This can extend work that you did in assignment 2, though add-1 smoothing is not required. In particular, given

a value for n , define

$$P(w_i|w_{i-1}, \dots, w_{i-n+1}) = \begin{cases} \frac{C(w_i, \dots, w_{i-n+1})}{C(w_i, \dots, w_{i-n+2})} & , \text{if } C(w_i, \dots, w_{i-n+1}) > 0 \\ \alpha \cdot P(w_i|w_{i-1}, \dots, w_{i-n+2}) & , \text{otherwise.} \end{cases}$$

Start by setting $\alpha = 0.4$. During training, replace all words appearing only once with an `<UNK>` tag and map unknown words during testing to this tag.

- (e) (5 pts) Using the same train/test split as part (c), train two separate trigram ($n = 3$) language models with backoff, one only on positive reviews and one only on negative reviews. For each test review, compute perplexity under both models and use this to predict a class label. Discuss the performance of this classifier and compare the approach against that in (c).
2. Recall the analogy task in the context of word embeddings. Given word representations **a**, **b**, and **c**, the goal is to identify a word **d** such that **a** : **b** :: **c** : **d** (“**a** is to **b** as **c** is to **d**”). A simple but surprisingly effective approach to solving this problem is to perform some vector arithmetic:

$$\mathbf{b} - \mathbf{a} + \mathbf{c} \approx \mathbf{d}$$

In particular, **d** is predicted to be the nearest word to $\mathbf{b} - \mathbf{a} + \mathbf{c}$ (ignoring **a**, **b**, and **c** themselves) with respect to cosine similarity.

For this problem, use the analogy dataset `questions-words.txt` available on Canvas and the pre-trained embeddings available in [Gensim](#) (pretrained models are mentioned [here](#)). Unless otherwise noted, ignore analogies containing words that are not present in the embedding vocabulary.

Be aware that code may take several minutes to finish running for each piece of this problem!

- (a) (5 pts) Evaluate the `glove-wiki-gigaword-50` and `glove-wiki-gigaword-100` pretrained embeddings on the analogy task using the vector arithmetic described above. The `most_similar` function may be helpful. Produce a table like the one below with accuracies for each category and accuracy overall. What do you observe?

	<code>glove-wiki-gigaword-50</code>	<code>glove-wiki-gigaword-100</code>
capital-common-countries		
capital-world		
currency		
city-in-state		
family		
gram1-adjective-to-adverb		
gram2-opposite		
gram3-comparative		
gram4-superlative		
gram5-present-participle		
gram6-nationality-adjective		
gram7-past-tense		
gram8-plural		
gram9-plural-verbs		
Overall		

As an aside, [GloVe](#) produces embeddings that are similar to those produced by Word2Vec but using a different approach.

- (b) (5 pts) Design and implement a simple feedforward neural network for the analogy task. For an analogy $\mathbf{a} : \mathbf{b} :: \mathbf{c} : \mathbf{d}$, your model should take the vectors $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ as input and produce a prediction for $\mathbf{d}$ as output. Describe the architecture that you choose, do not worry about tuning the architecture yet.

Split the data into training, validation, and test sets. Describe your approach carefully. Train your model and report accuracies on the **validation set**. Do not consider the test set yet.

	glove-wiki-gigaword-50	glove-wiki-gigaword-100
capital-common-countries		
capital-world		
currency		
city-in-state		
family		
gram1-adjective-to-adverb		
gram2-opposite		
gram3-comparative		
gram4-superlative		
gram5-present-participle		
gram6-nationality-adjective		
gram7-past-tense		
gram8-plural		
gram9-plural-verbs		
Overall		

- (c) (10 pts) Explore the effects of network depth, hidden layer width, and choice of non-linear activation function on model performance. Describe your experimental setup process. For the model with best performance on the validation set, report accuracies on the test set and discuss the results. Finally, compare these accuracies against the performance of the vector arithmetic approach on the test set (note that you may need to rerun part (a) using a subset of the data).

	glove-wiki-gigaword-50	glove-wiki-gigaword-100
capital-common-countries		
capital-world		
currency		
city-in-state		
family		
gram1-adjective-to-adverb		
gram2-opposite		
gram3-comparative		
gram4-superlative		
gram5-present-participle		
gram6-nationality-adjective		
gram7-past-tense		
gram8-plural		
gram9-plural-verbs		
Overall		